

HTTP Redirect Headers

draft-hardt-httpbis-redirect-headers

Dick Hardt (Hellō) & Sam Goto (Google)

IETF 125 Shenzhen — March 2026

Agenda

- Problem: query parameter leakage in protocol redirects
- The solution: move parameters to new HTTP Redirect-* headers
- OAuth flow: before and after
- Incremental deployment — no coordination required
- Redirect-Origin and Redirect-Path details
- Security and privacy considerations
- Next steps and open questions

Problem Statement

- Authentication protocols (OAuth, OIDC, SAML) pass sensitive parameters via URL query strings during browser redirects
- Authorization codes, tokens, and session identifiers leak through:
 - Browser history, server logs, URL sharing, malicious JS on the page
- Extensions can observe full request URLs and page content
- Origin verification (Referer) is unreliable — stripped by privacy tools, proxies, browsers

The Solution: Three New HTTP Headers

- **Redirect-Query** — carries parameters in headers instead of URLs
- **Redirect-Origin** — browser-attested sender origin (cannot be spoofed or stripped)
- **Redirect-Path** — optional path-specific origin verification

How It Works

- Server sets Redirect-Query and optional Redirect-Path in a 30x redirect response
- Browser detects Redirect-* headers and forwards Redirect-Query to the target, adding Redirect-Origin
- Receiving server gets parameters + verified sender origin in headers, not URLs

OAuth Flow: Before

- OAuth client redirects to AS with parameters in URL
- AS redirects back with `?code=Sp1x10Be&state=123` in URL
- Authorization code exposed in browser history, logs, and page JavaScript (XSS, injected scripts, third-party tags, extensions)

OAuth Flow: After

- OAuth client sets Redirect-Query header in 30x redirect response
- Browser forwards parameters in headers, adds Redirect-Origin
- AS responds with code in Redirect-Query header only
- Authorization code never appears in any URL

Incremental Deployment

"This requires browsers, clients, AND servers — will it ever get adopted?"

- Deployment does **not** require coordination — each party adopts independently
- Browser and AS only change behavior when previous party has support
 - **OAuth clients** start sending params in both URL and Redirect-Query header
 - **Browsers** detect Redirect-* headers in redirect responses → forward them with Redirect-Origin
 - **AS** receives Redirect-Query → knows client AND browser support it → responds with header only
- Starts working as soon as all three parties have support
- Graceful degradation — unsupported parties just see normal URL-based redirects

Redirect-Origin: Browser-Attested Sender Verification

- Set only by the browser — cannot be spoofed by scripts
- Always present when Redirect-Query is used
- Enables receiving party to verify who sent the redirect
- Mitigates mix-up attacks (RFC 9700 §4.4) and authorization code injection (RFC 9700 §4.5)
- Unlike Referer: always present, always accurate

Redirect-Path: Path-Specific Origin Verification

- Multiple apps or tenants may share a single origin under different path prefixes
- Server requests a path prefix be included in Redirect-Origin (e.g., `Redirect-Path: "/mobile/"`)
- Browser verifies the server is actually serving from that path before including it
 - ✓ Valid → Redirect-Origin includes the path: `https://app.example/mobile/`
 - ✗ Invalid → path ignored, origin-only: `https://app.example/`
- Receiving party can verify not just the origin, but **which path** within that origin

Security Considerations

- Authorization codes never appear in URLs
- Browser-enforced: JavaScript and extensions cannot access Redirect-* headers
- Top-level navigation only — not processed for subresource requests
- Servers **MUST** exclude Redirect-Query from logs — carries sensitive credentials

Privacy Considerations

- Redirect-Origin reveals the redirecting origin (like Referer, but reliable)
- Trade-off: security (sender verification) over origin hiding
- Unlike Referer, users/tools cannot strip Redirect-Origin when Redirect-Query is present

Open Questions 1/3

- **Q1: IETF vs. WHATWG?**
 - Feature for OAuth -> people that care are in IETF
 - Browser behavior requirements need coordination with Fetch specs
- **Q2: Why not form_post?**
 - form_post exposes parameters to page JS and extensions;
 - most deployments use redirects
 - adopting Redirect Headers is a library update
 - changing GET to POST which requires infra and request routing change

Open Questions 2/3

- **Q3: Why not Structured Fields Dictionary?**
 - Redirect-Query uses a string so recipients parse it the same way as URL query strings; avoids a new encoding
- **Q4: Relationship to Origin header?**
 - Redirect-Origin has different semantics: browser-attested, always present with Redirect-Query, includes optional path, cannot be set by scripts

Open Questions 3/3

- **Q5: Why not Sec- prefix?**
 - Sec-* headers are forbidden from being set by JS, but they are request headers;
 - Redirect-Query is set by servers in responses, then forwarded by the browser
- *Perhaps*
 - Sec-Redirect- request headers (browser) and
 - Redirect- response headers
- **Q6: Header size limits?**
 - form_post is used for large responses
 - will this exceed header size limits in deployments?

Next Steps

- Feedback from HTTPBIS working group and adoption
- Browser vendor engagement (Chromium has expressed interest)
- OAuth client library engagement (Better Auth expressed support)
- Authorization server / identity provider engagement

Other Questions / Discussion

- dick.hardt@gmail.com
- draft-hardt-httpbis-redirect-headers
- <https://github.com/dickhardt/redirect-headers>